Output Options

RSpec aims to provide *actionable* output—that is, output that helps you decide what to do next. To that end, it supports a number of options to make the output useful for specific situations:

config.warnings = true

Enables Ruby's warnings mode, like the rspec --warnings flag we discussed earlier. This helps you catch some mistakes (such as method redefinitions and variable misspellings) but may report tons of extra warnings in third-party code, unless you use something like ruby_warning_filter to cut some of the noise.[6]

config.profile_examples = 2

RSpec will measure how long each spec took and print the given number of slowest examples and groups (two, in this case). This is helpful for keeping your test suite fast.

When an expectation fails, RSpec prints the backtrace showing the chain of method calls all the way from your spec to the lowest-level code. RSpec excludes its own stack frames from this list. You can also exclude other libraries or files from the backtrace:

config.backtrace_exclusion_patterns << /vendor/

Excludes any lines from the backtrace matching the given regular expressions; for example, lines containing the text vendor.

config.filter_gems_from_backtrace :rack, :sinatra

Excludes stack frames from specific libraries; here, we won't see calls from inside the rack and sinatra gems.

If you ever need more detail, you can get the full backtrace (including stack frames from RSpec and any others you have configured it to ignore) by passing --backtrace on the command line.

Nearly all of RSpec's output is customizable using a *formatter*. As we've previously discussed, you can specify a formatter on the command line using the --format or -f option. You can also add a formatter in an RSpec.configure block:

09-configuring-rspec/03/rspec_configure.rb

```
RSpec.configure do |config|
  # You can use the same formatter names supported by the CLI...
  config.add_formatter 'documentation'
```

6. https://github.com/semaperepelitsa/ruby_warning_filter